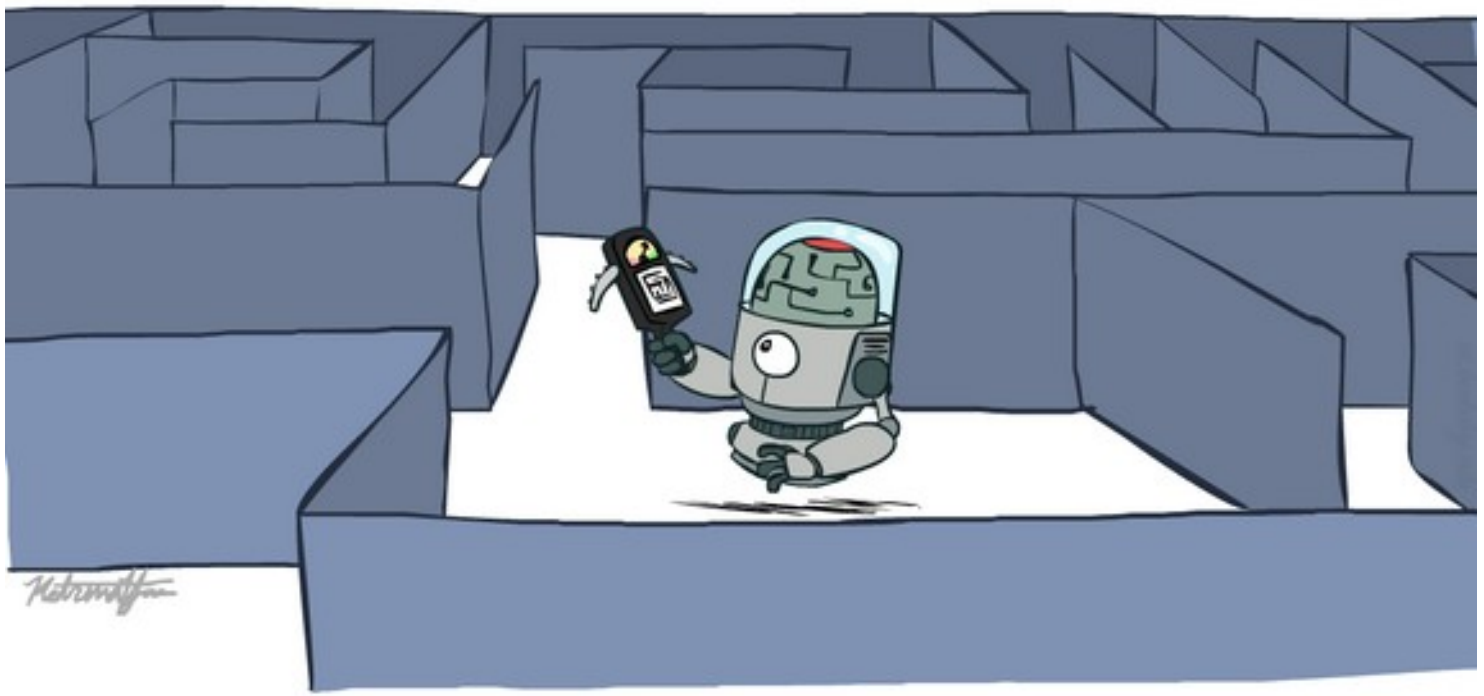
KU LEUVEN

Fundamentals of Artificial Intelligence

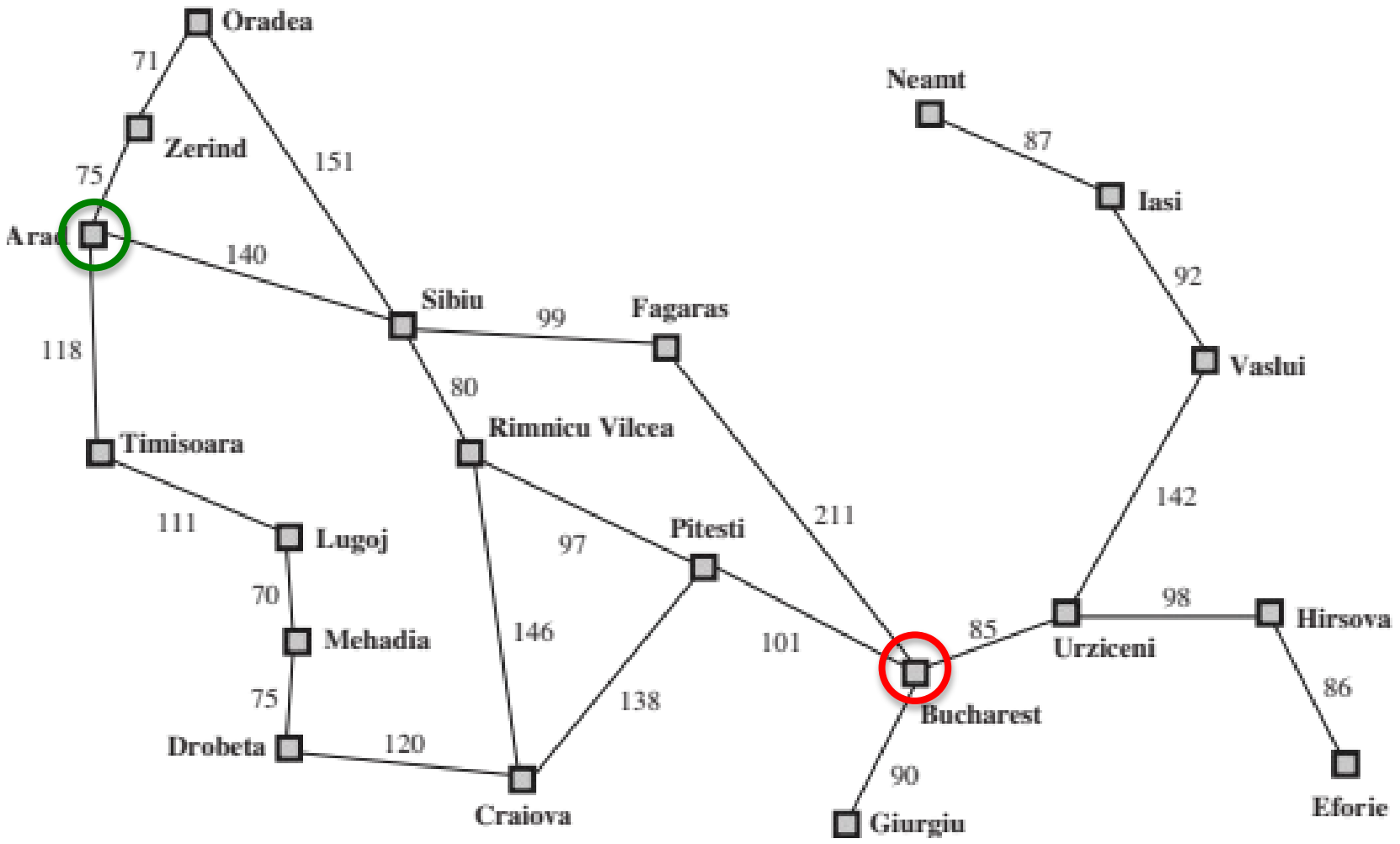


Prof. Tias Guns

Lecture 04, 2024-25
Informed Search

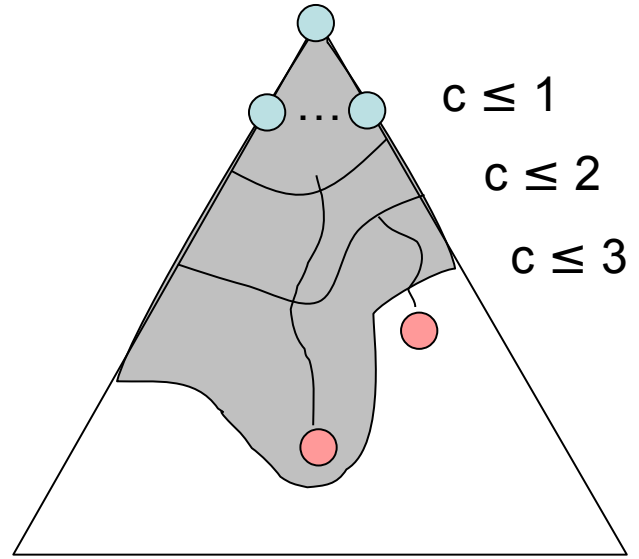
Based on the slides of CS188 @ Berkeley and Prof. De Raedt's AI course

Example: route-finding in Romania



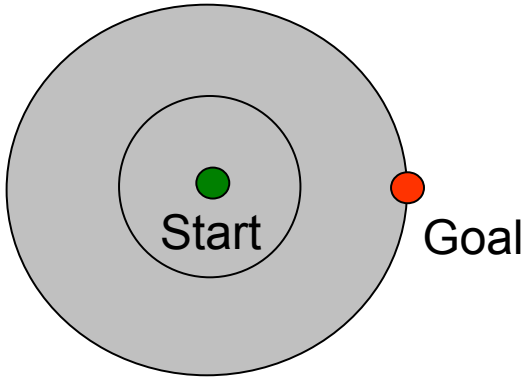
Uniform Cost Search

- Strategy: expand lowest path cost



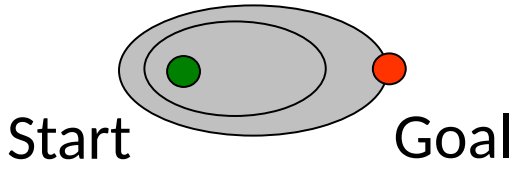
- The good: UCS is complete and optimal!

- The bad:
  - Explores options in every “direction”
  - No information about goal location

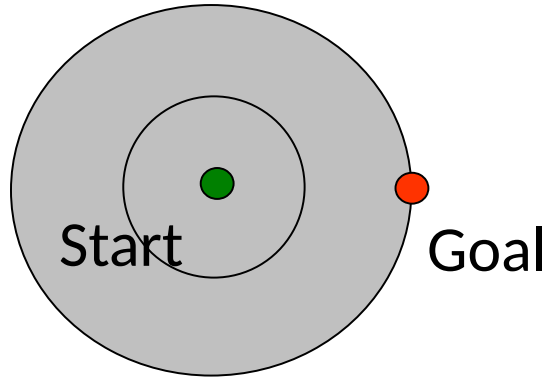


What we would like to have happen

Guide search ***towards the goal*** instead of ***all over the place***



Informed

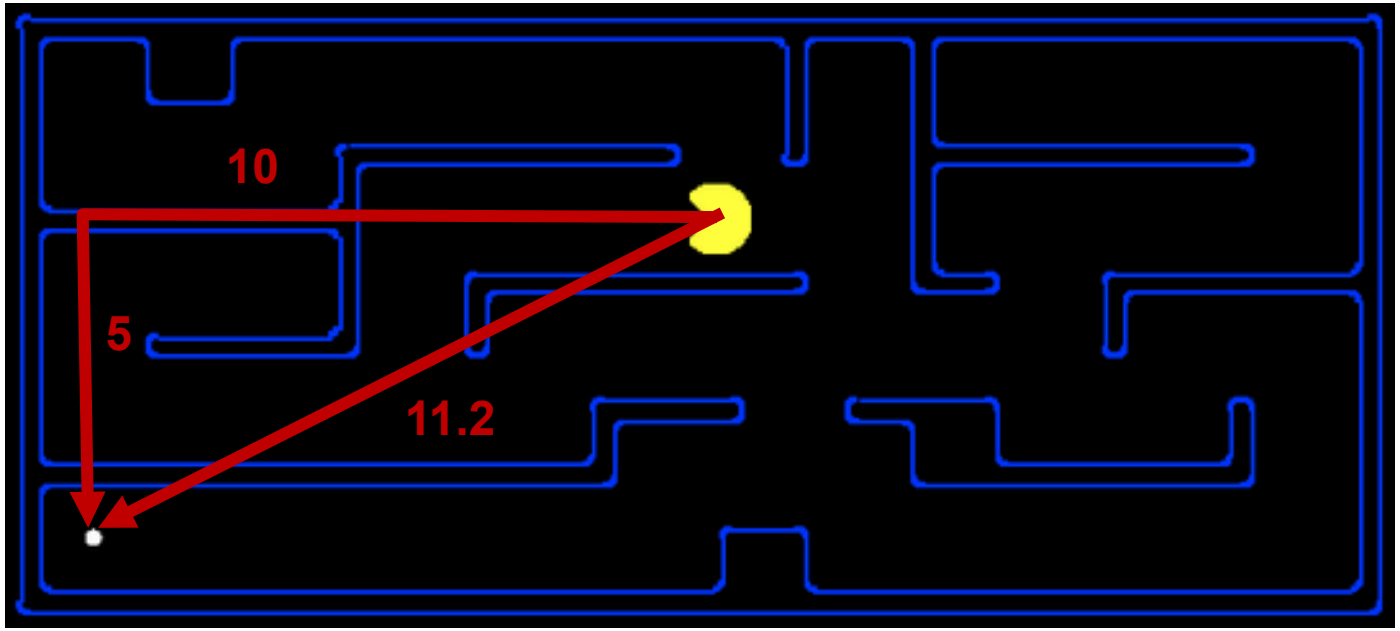


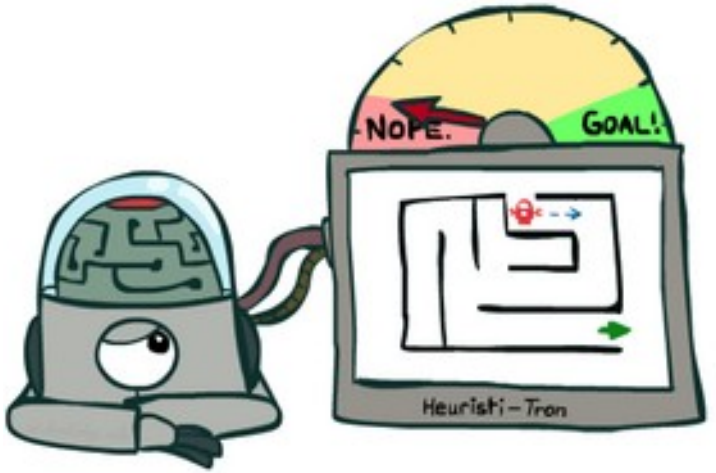
Uninformed

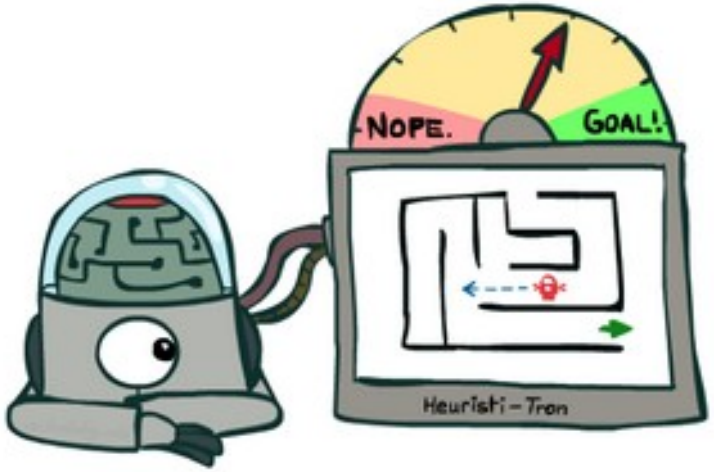
Search Heuristics

- A heuristic is:
  - A function that *estimates* how close a state is to a goal
  - Designed for a particular search problem
  - Path planning?

    Manhattan distance
    Euclidean distance





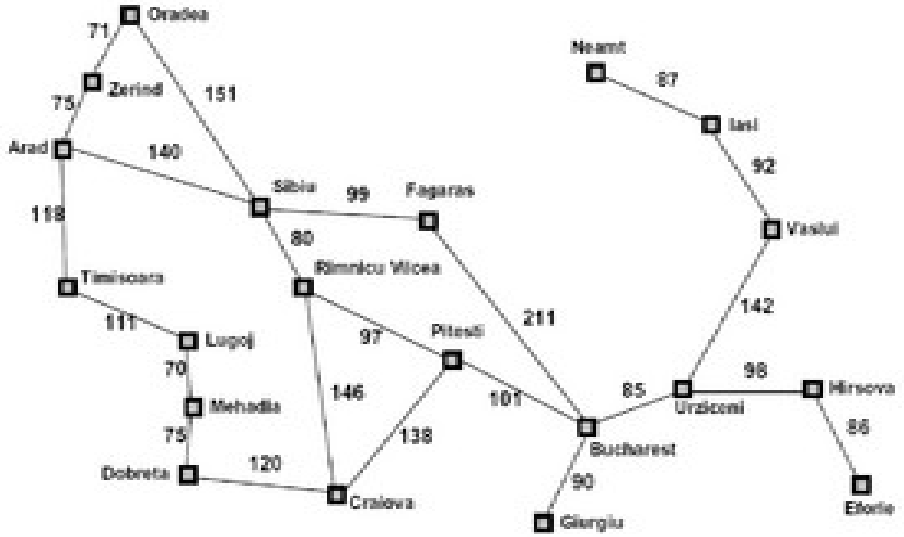


Greedy Best-First Search

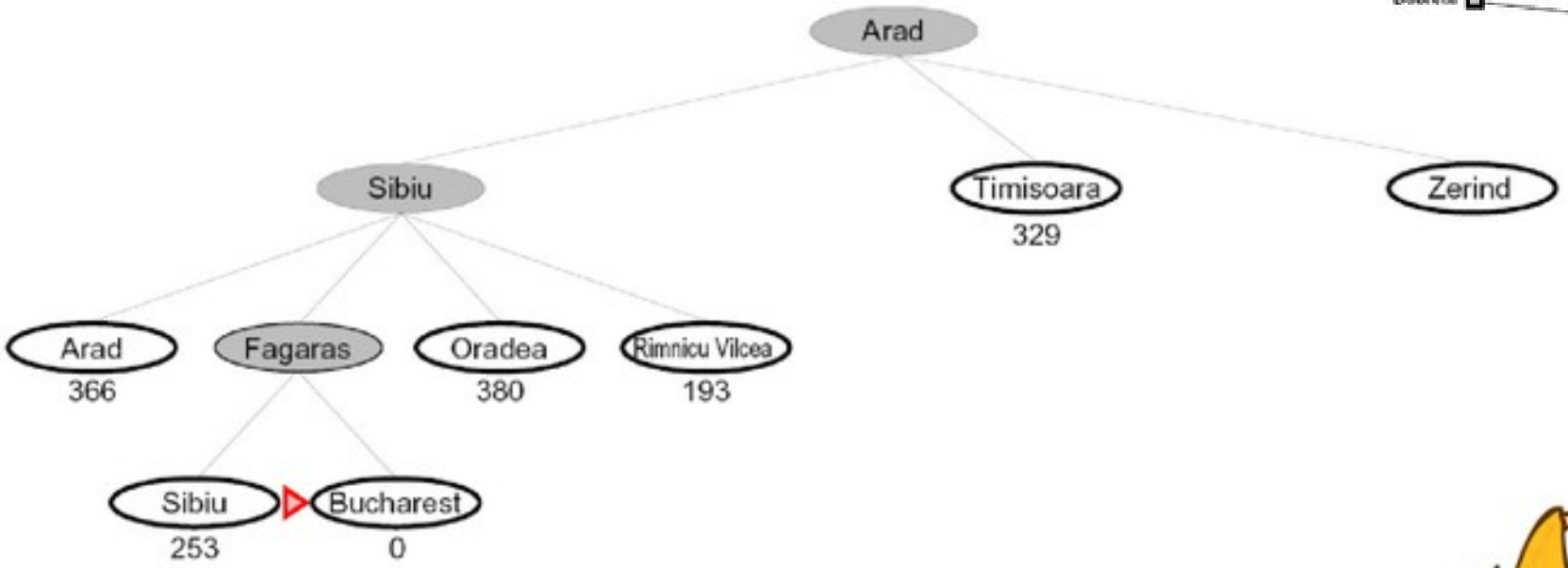


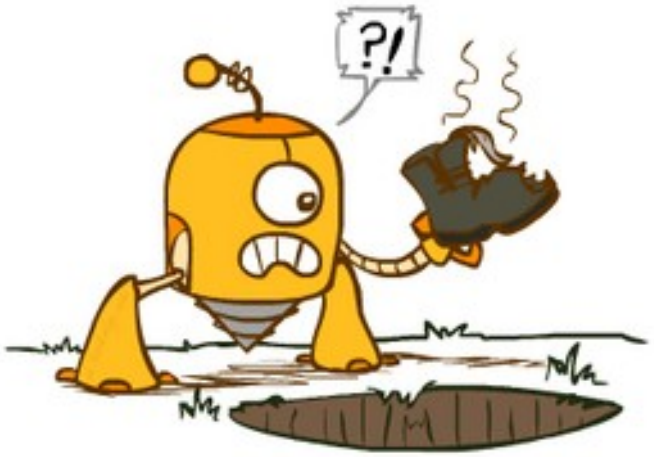
Greedy Best-First Search

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



- Expand the node that seems closest…
  (Euclidean distance to goal)



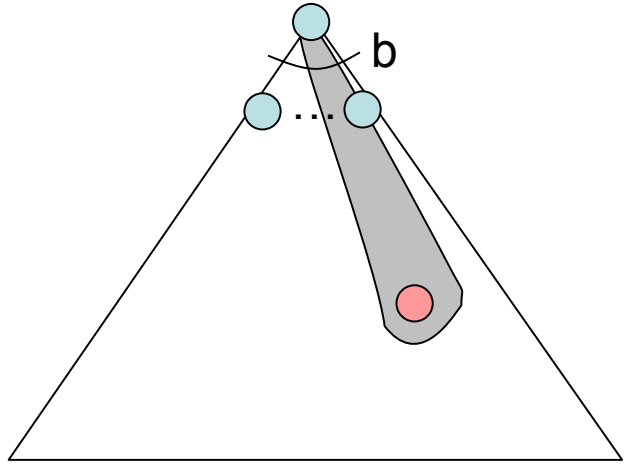


- Is it optimal?

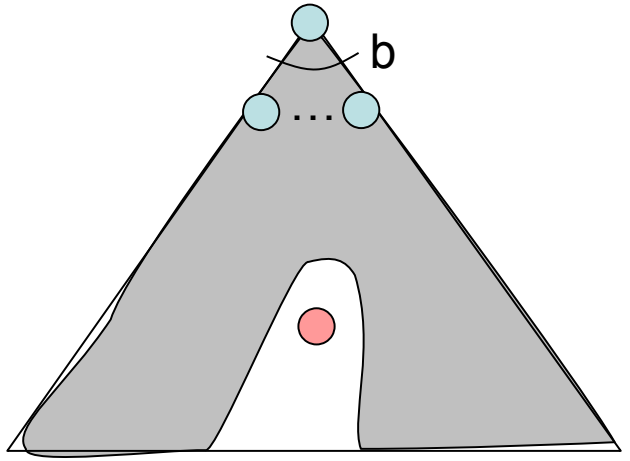
No. Resulting path to Bucharest is not the shortest!

Greedy Best-First Search (GBFS)

- Strategy: expand a node that you think is closest to a goal state
  - Heuristic: estimate of distance to nearest goal for each state

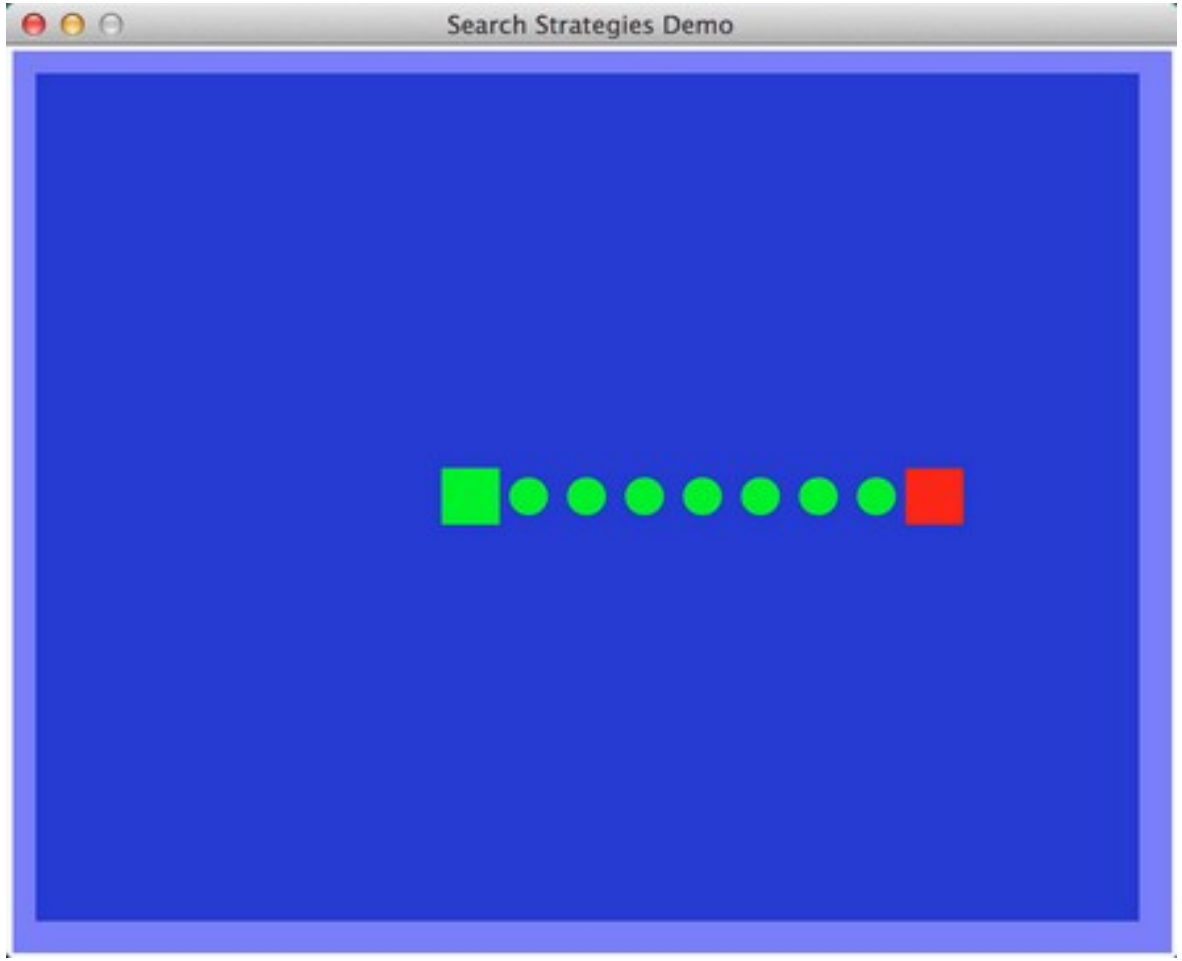


- A common case:
  - Greedy best-first takes you straight to *a* goal



- Worst-case: like a badly-guided DFS

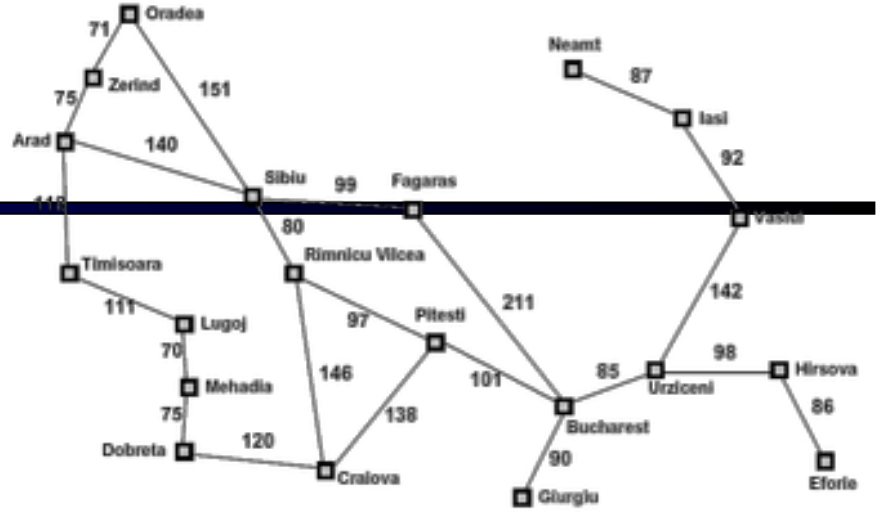
Video of Demo Contours GBFS (Empty)



Video of Demo Contours GBFS (Pacman Small Maze)



Beam Search



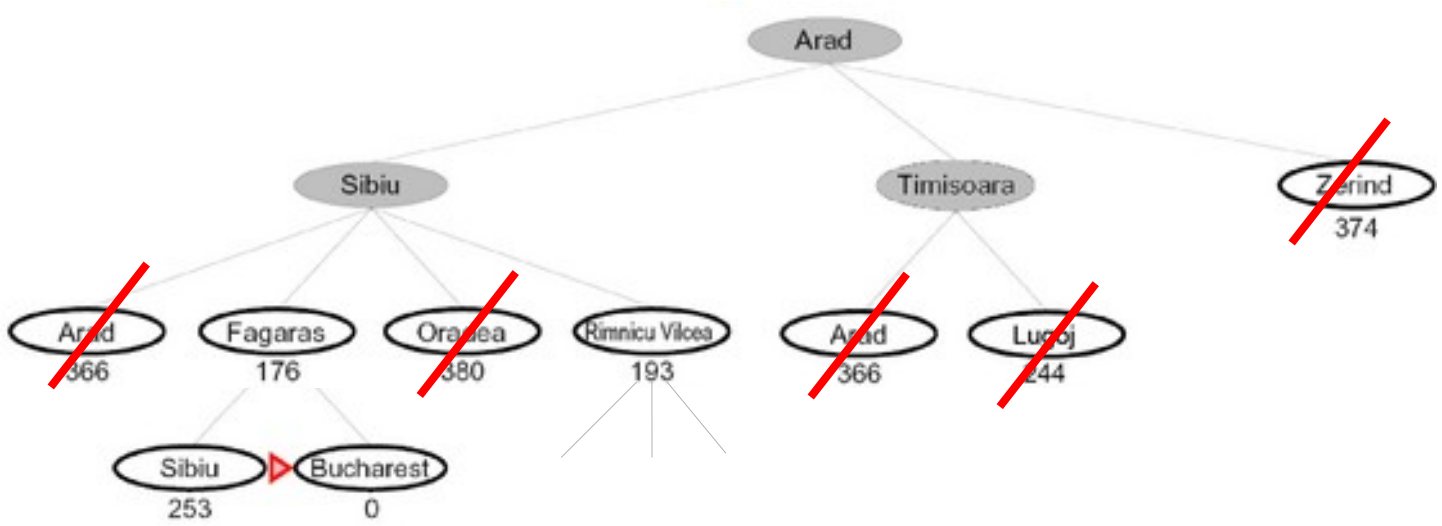
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Variant of breadth-first search and greedy best-first

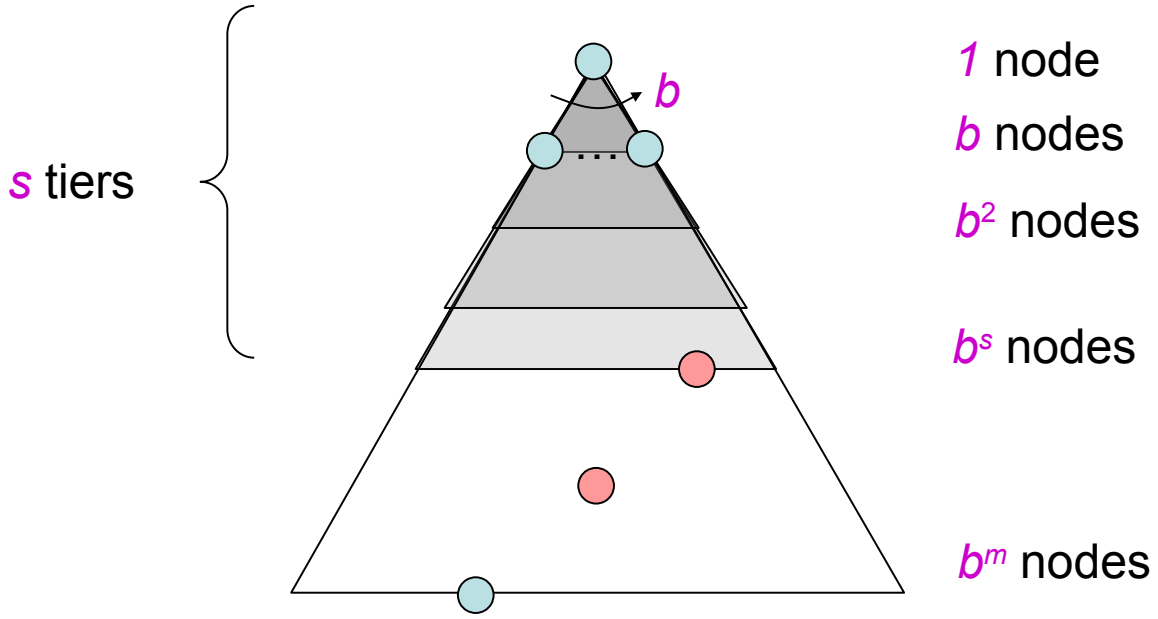
- just like Breadth-first search: expand all nodes that are at the same depth,
  but after every *level*: only keep the *k* best paths on the frontier, remove all others

- just like Greedy: choose which *k* paths to keep based on their heuristic value



Beam Search Properties, beam size *k*

- How much space does the frontier take?
  - Queue has always size *k*, but expansion requires size *O*(*kb*)
- What nodes does BFS expand?
  - If a solution is found: *O*(*skb*)
  - otherwise: *O*(*mkb*)
- Is it complete?
  - NO! you may throw away a needed path
- Is it optimal?
  - NO!



- '(greedy) hill-climbing search': K = 1 (stick to 1 path, no 'backtracking')

Informed search...

- Greedy best-first search is not optimal…

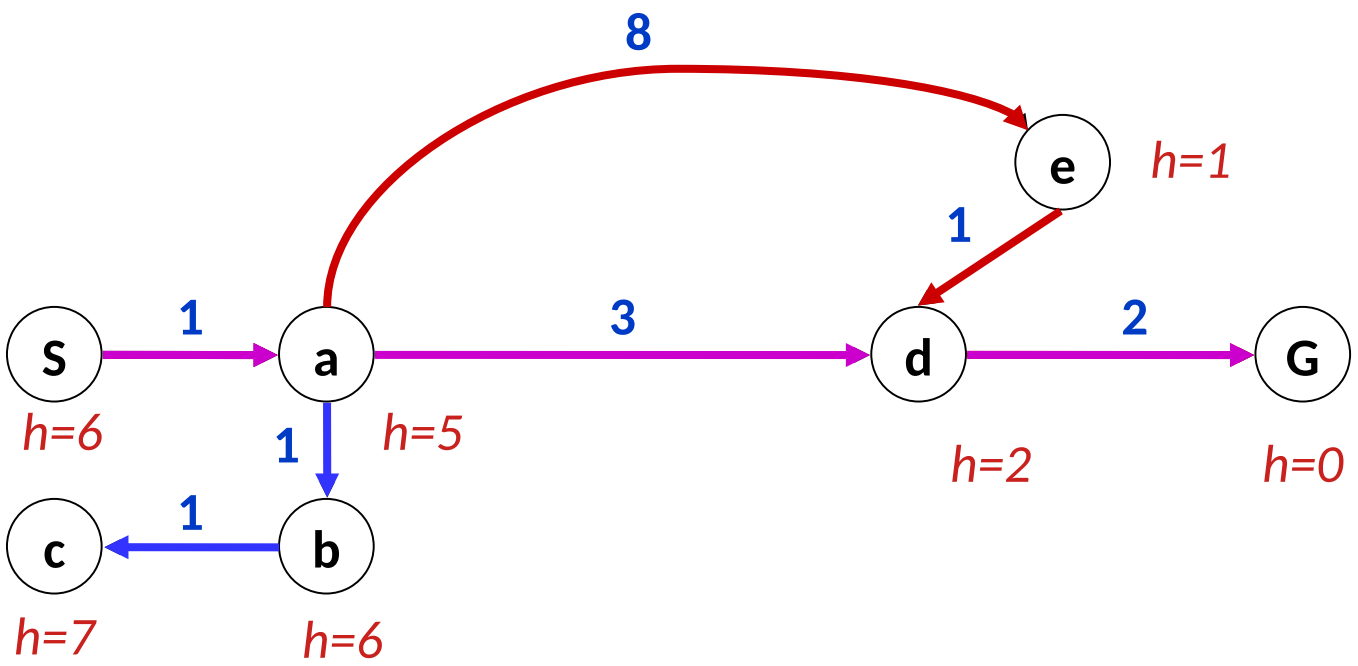
- Can we do **informed search**, like GBFS, but with optimality guarantees **like UCS**?

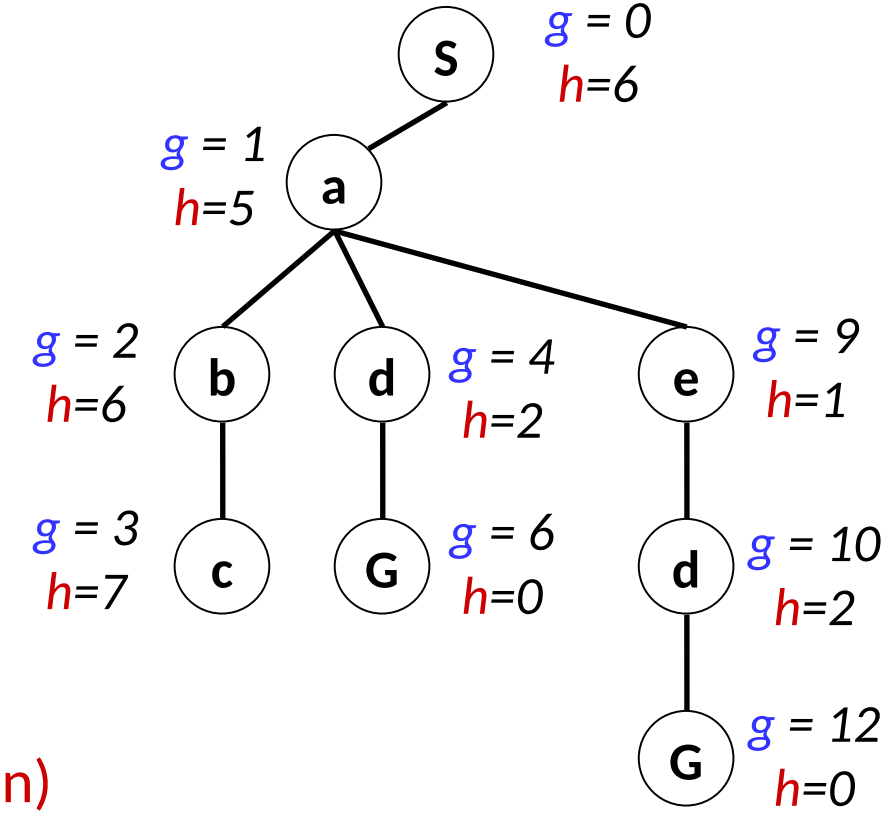
A* Search



Combining UCS and Greedy

- Uniform-cost orders by path cost, or *backward cost* g(n)
- Greedy orders by goal proximity, or *forward cost* h(n)





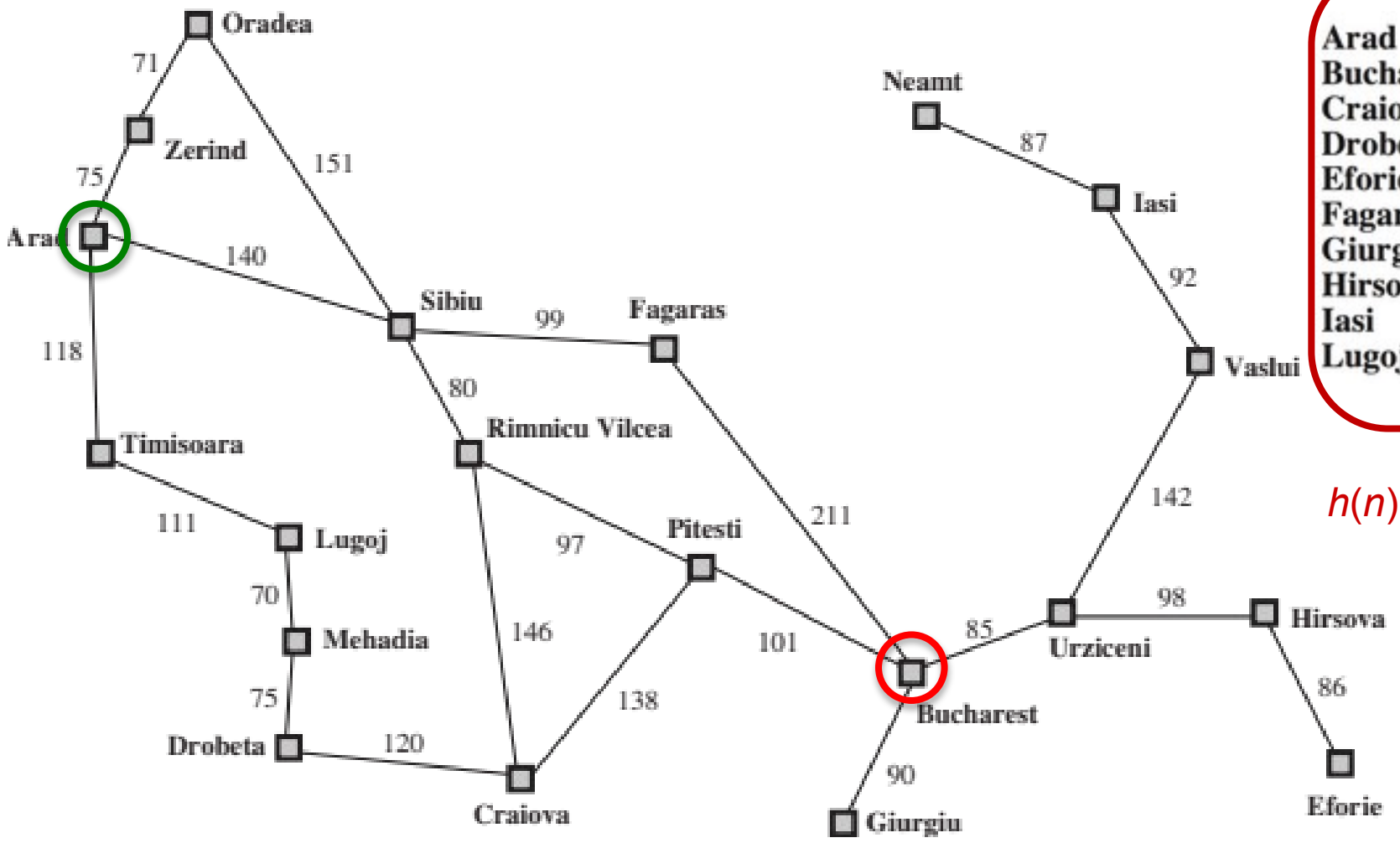
- A* Search orders by the sum: f(n) = g(n) + h(n)

Example: Teg Grenager

A*: the core idea

- Expand a node n most likely to be on an optimal path
- Expand a node n with lowest value of $g(n) + h^*(n)$
  - $g(n)$ is the cost from root to n
  - $h^*(n)$ is the optimal cost from n to the closest goal
- We seldom know $h^*(n)$ but might have a heuristic approximation $h(n)$
- A* = tree search with priority queue ordered by $f(n) = g(n) + h(n)$

Example: route-finding in Romania

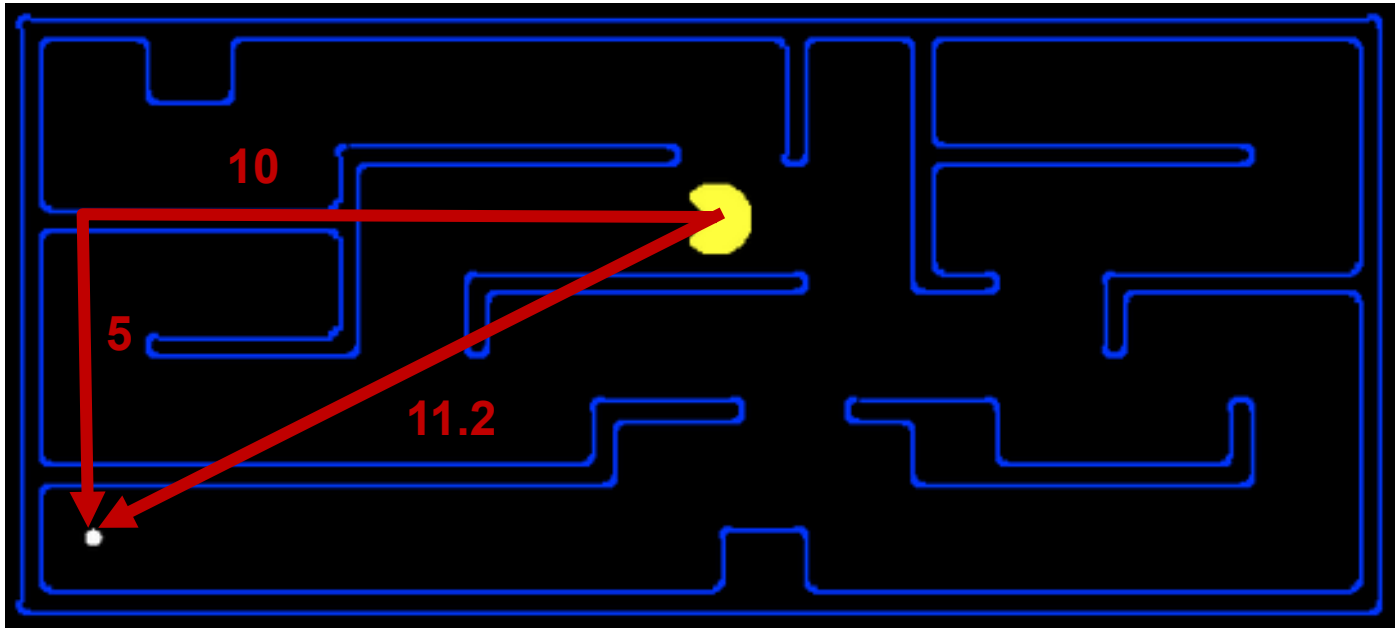


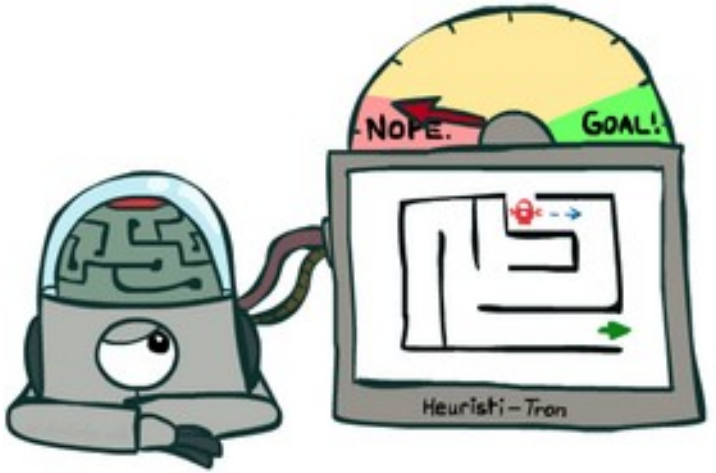
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

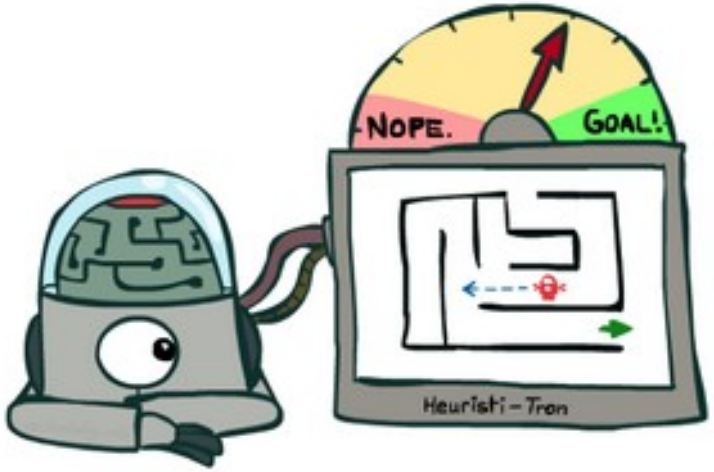
$h(n)$ = Euclidean distance to Bucharest

Example: pathing in Pacman

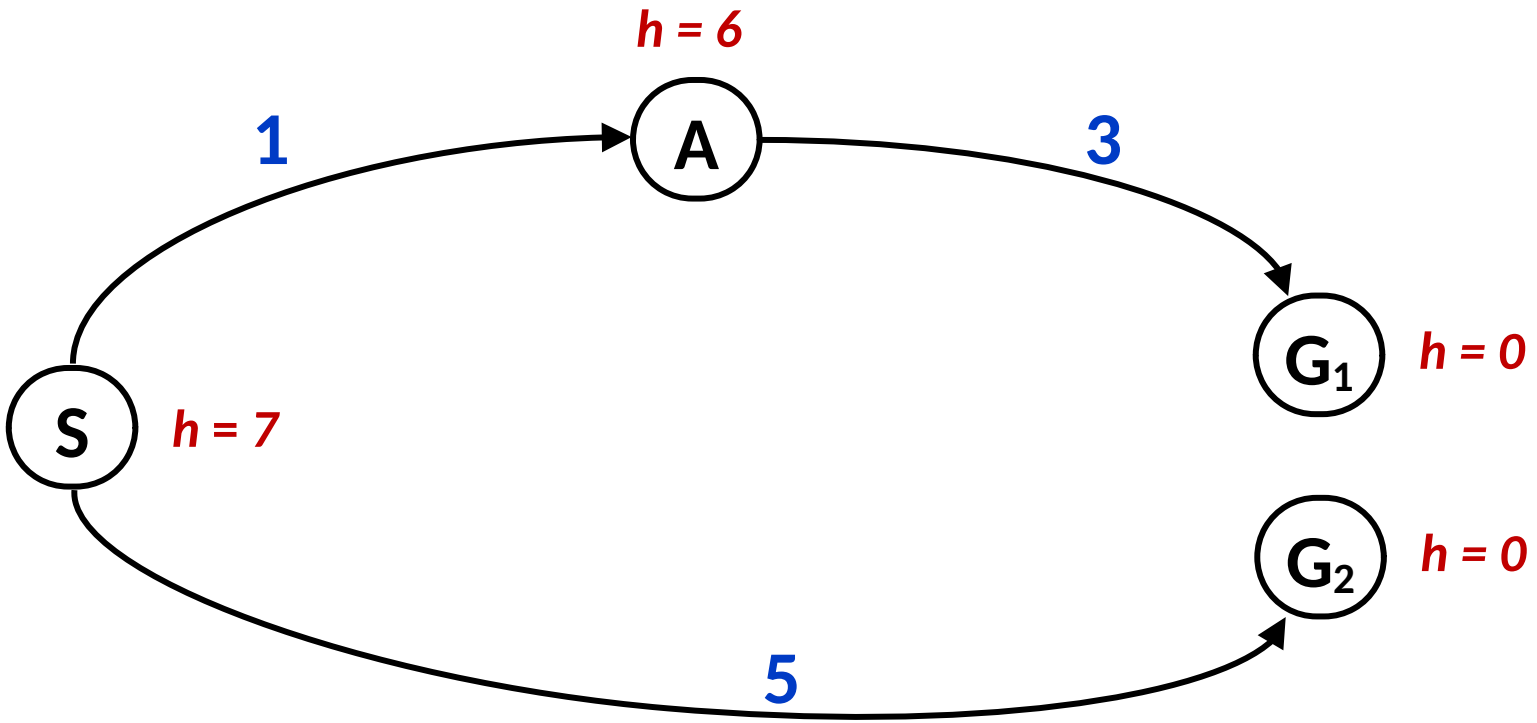
- $h(n)$ = Manhattan distance = $|\Delta x| + |\Delta y|$
- Is Manhattan better than straight-line distance?







Is A* Optimal?



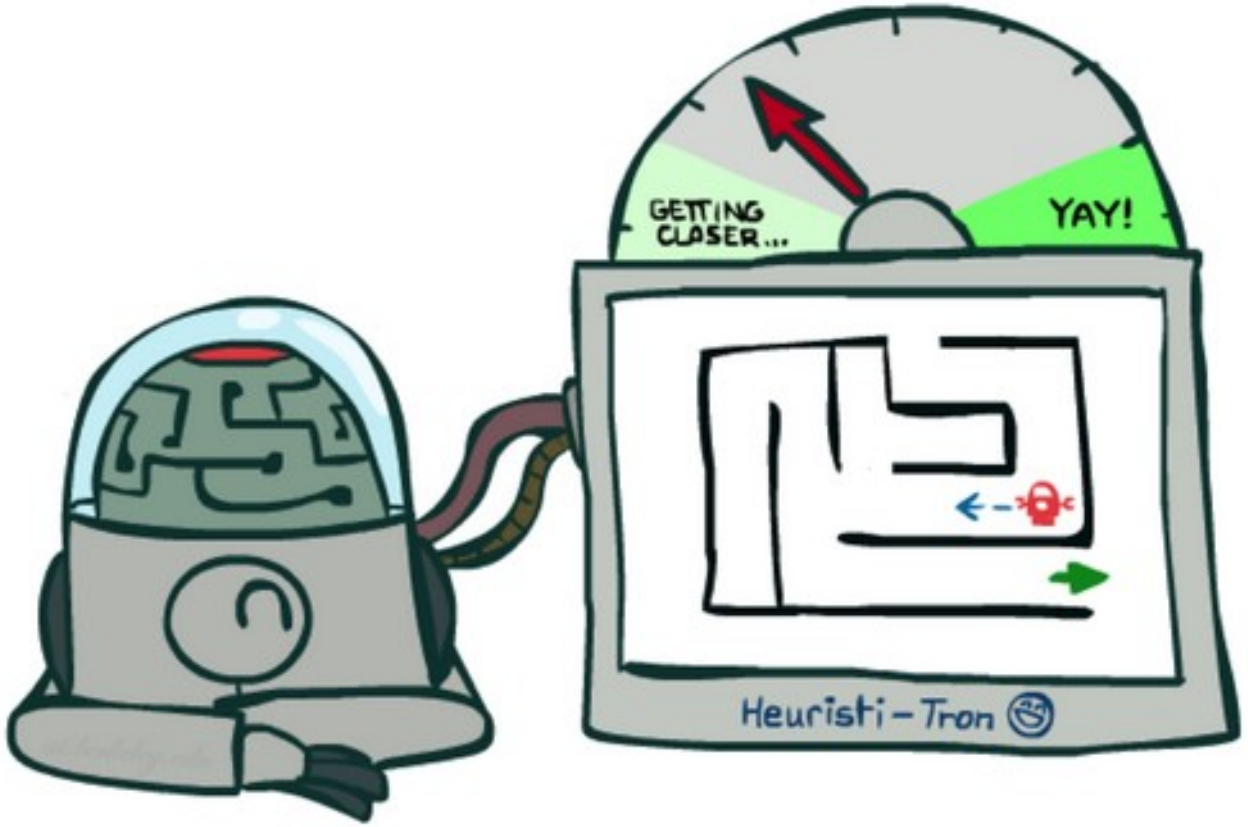
[S:0+7=7]
[SA:1+6=7,
SG_2:5+0=5]
Expand and stop with SG_2=5

but SAG_1=4?

What went wrong?

- ***Actual*** SAG_1 is better then the ***estimated*** SA
- ***Actual*** bad solution cost SG_2=5 < ***estimated*** good solution cost SA=7

Admissible Heuristics



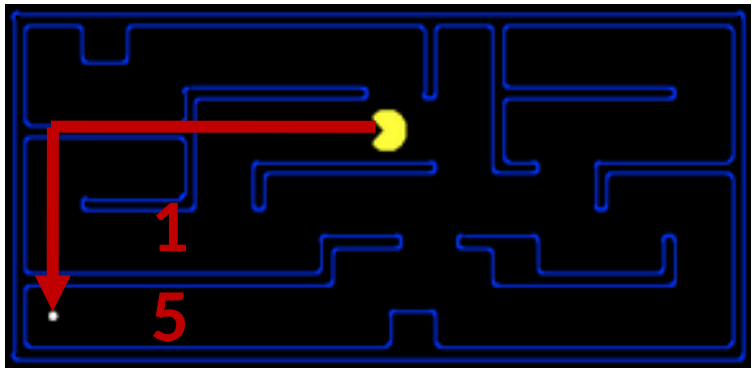
Admissible Heuristics

- A heuristic h is *admissible* if:

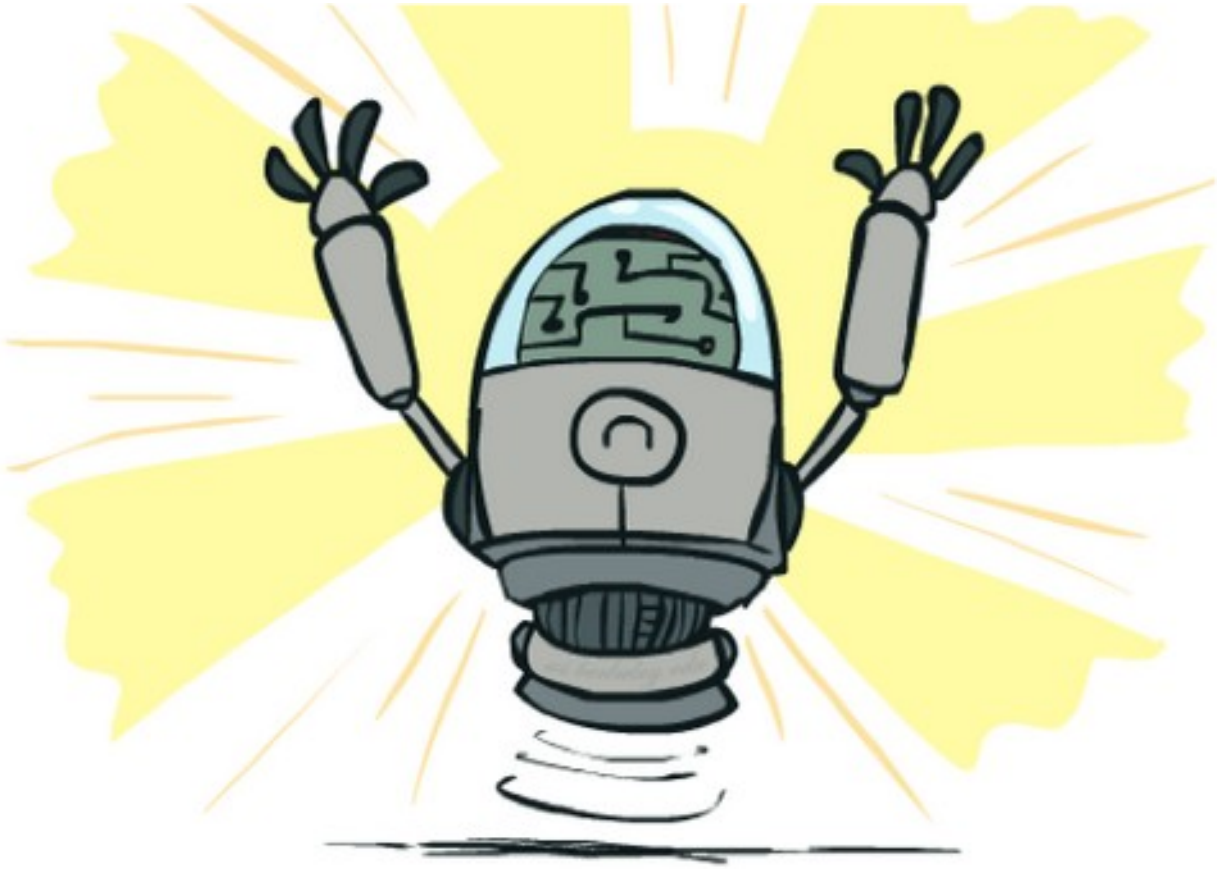
$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to a nearest goal

- Only optimistic estimates!
- Example:



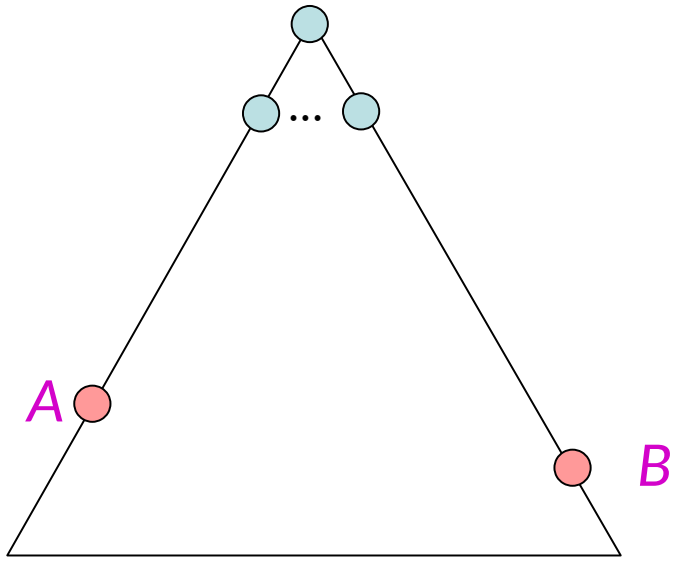
Optimality of A* Tree Search



Optimality of A* Tree Search (no loop br.)

Given:

- *A* is an optimal goal node
- *B* is a suboptimal goal node
- *h* is admissible



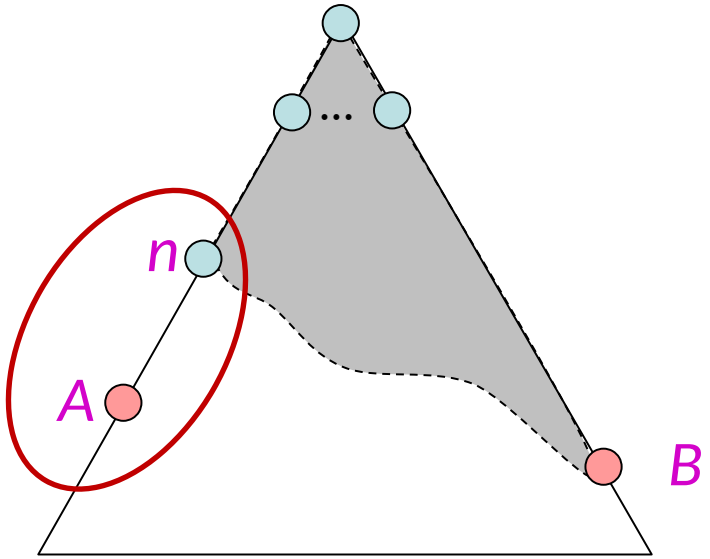
Claim to prove:

- *A* will be chosen for expansion before *B* in A* tree search

Optimality of A* Tree Search: Blocking

Proof:

- Imagine *suboptimal* *B* is on the frontier
- Some ancestor *n* of *A* is on the frontier, too (maybe *A* itself!)
- Claim: *n* will be expanded before *B*
  1. $f(n) \leq f(A)$

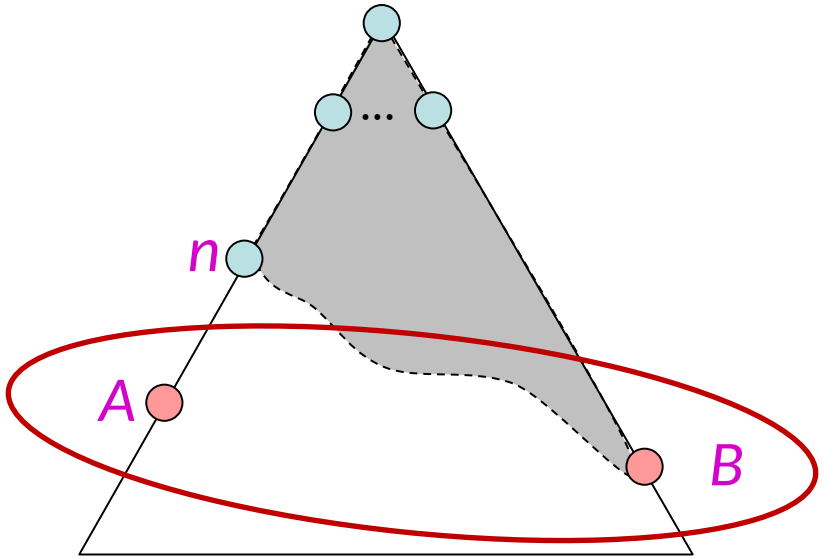


$f(n) = g(n) + h(n)$	Definition of *f*-cost
$g(n) + h(n) \leq g(n) + h^*(n)$	Admissibility of *h*
$f(n) \leq g(A)$	(rewrite)
$g(A) = f(A)$	$h = 0$ at a goal

Optimality of A* Tree Search: Blocking

Proof:

- Imagine *B* is on the frontier
- Some ancestor *n* of *A* is on the frontier, too (maybe *A* itself!)
- Claim: *n* will be expanded before *B*
  1. $f(n) \leq f(A)$
  2. $f(A) < f(B)$



$g(A) < g(B)$

$f(A) < f(B)$

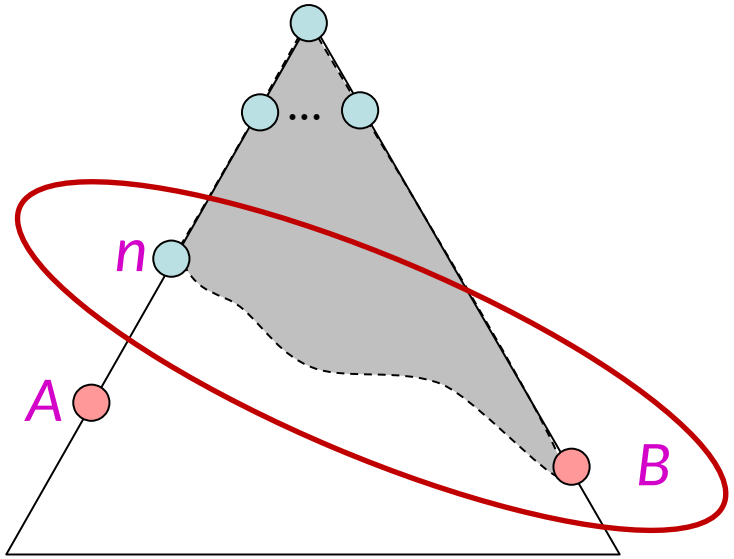
Suboptimality of *B*

$h = 0$ at a goal

Optimality of A* Tree Search: Blocking

Proof:

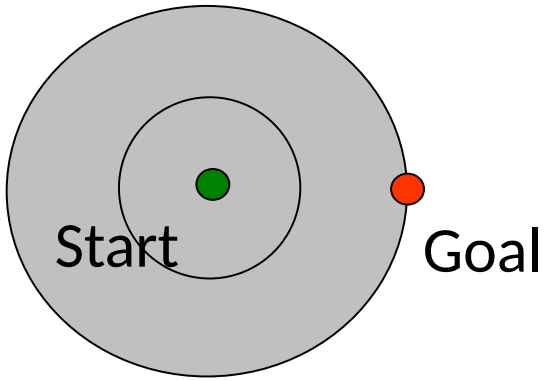
- Imagine *B* is on the frontier
- Some ancestor *n* of *A* is on the frontier, too (maybe *A* itself!)
- Claim: *n* will be expanded before *B*
  1. $f(n) \leq f(A)$
  2. $f(A) < f(B)$
  3. *n* is expanded before *B*
- All ancestors of *A* are expanded before *B*
- *A* is expanded before *B*
- **A* tree search is optimal**



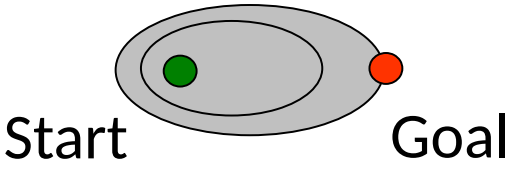
$$f(n) \leq f(A) < f(B)$$

UCS vs A* Contours

- Uniform-cost expands equally in all "directions"



- A* expands mainly toward the goal, but does hedge its bets to ensure optimality



Comparison



Uniform Cost (g)



Greedy
(h)



A* (g+h)

A* Applications

- Video games
- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Machine translation
- Speech recognition
- Protein design
- Chemical synthesis
- …



Creating Heuristics

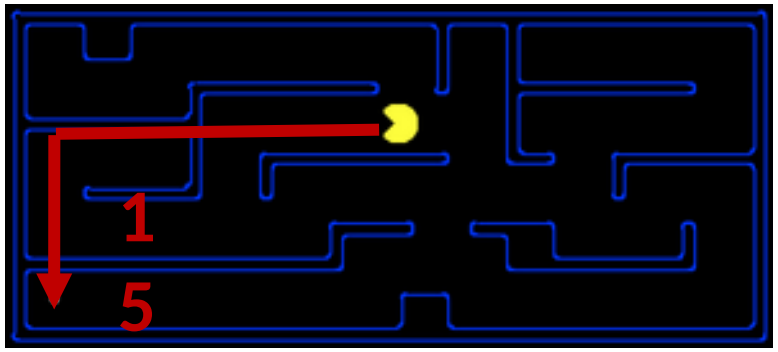


Creating Admissible Heuristics

- Often, admissible heuristics are solutions to ***relaxed problems***, where new actions are available

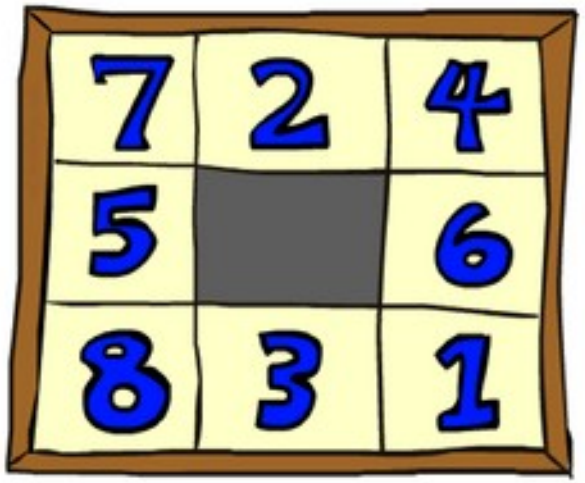


366

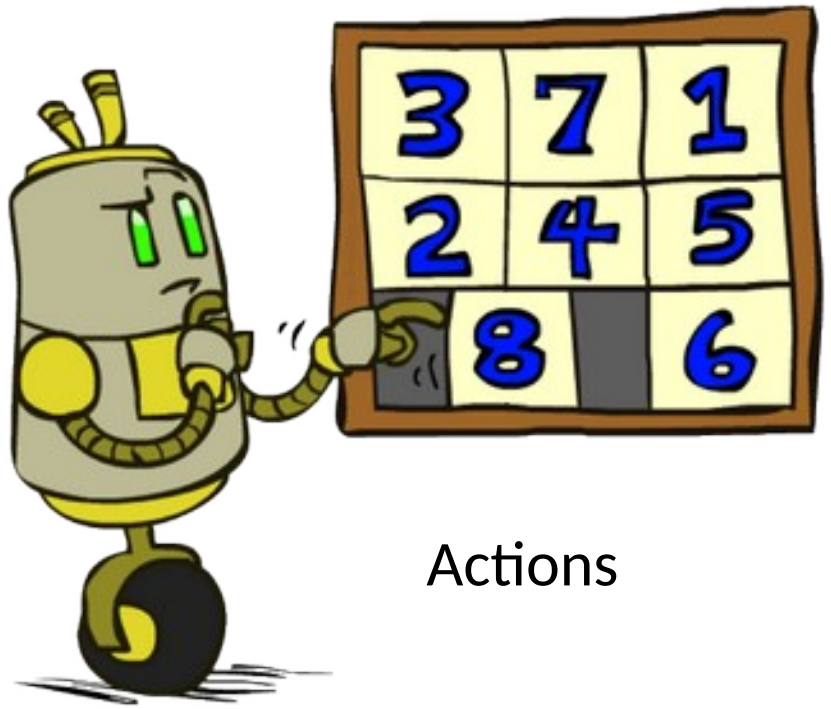


- Problem P_2 is a relaxed version of P_1 if $A_2(s) \supseteq A_1(s)$ for every s
- Theorem: $h_2^*(s) \leq h_1^*(s)$ for every s, so $h_2^*(s)$ is admissible for P_1

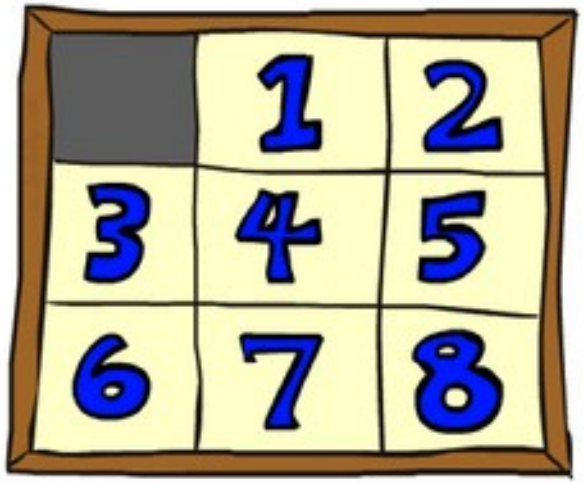
Example: 8 Puzzle



Start State



Actions



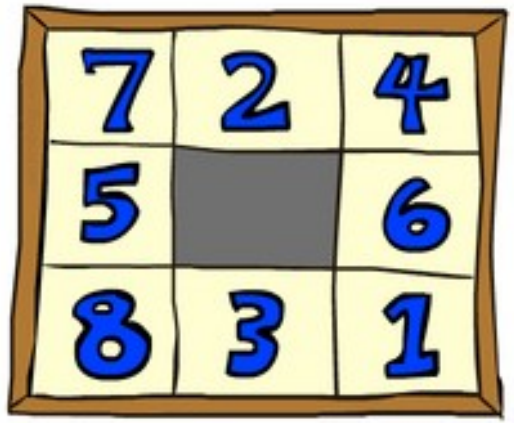
Goal State

- What are the states?
- How many states?
- What are the actions?
- What are the step costs?

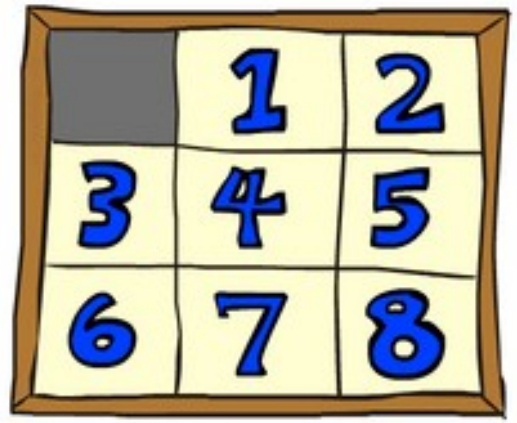
8 Puzzle I

- Heuristic: Number of tiles misplaced
- Why is it admissible?

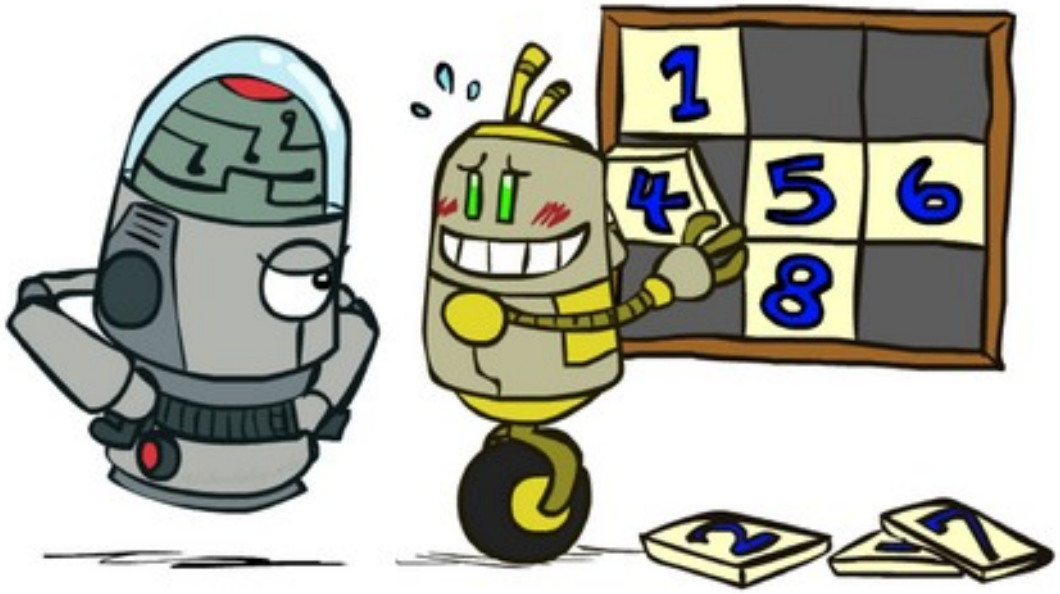
h(start) = 8



Start State



Goal State

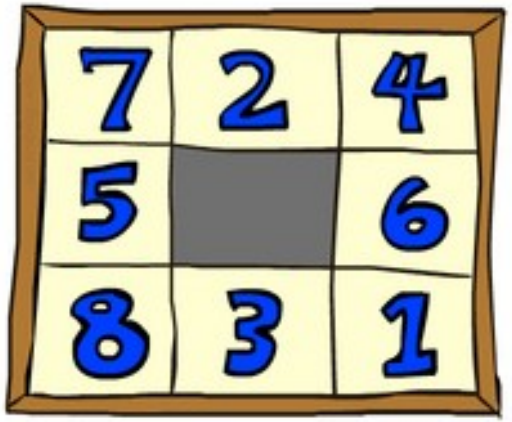


	Average nodes expanded when the optimal path has…		
	…4 steps	…8 steps	…12 steps
UCS	112	6,300	3.6×10^6
A*TILES	13	39	227

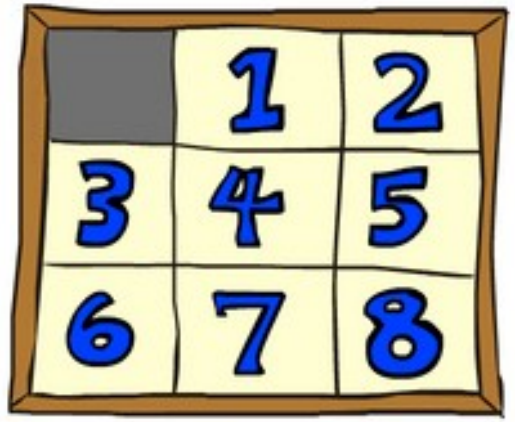
Statistics from Andrew Moore

8 Puzzle II

- What if we had an easier 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?
- Total *Manhattan* distance
- Why is it admissible?
- h(start) = 3 + 1 + 2 + … = 18



Start State

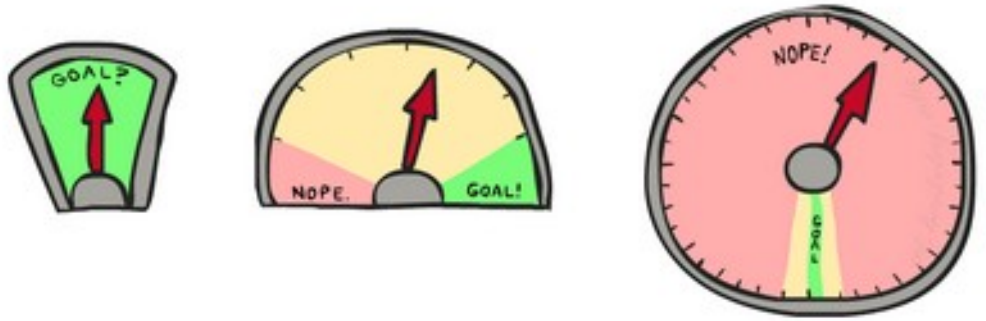


Goal State

	Average nodes expanded when the optimal path has…		
	…4 steps	…8 steps	…12 steps
A*TILES	13	39	227
A*MANHATTAN	12	25	73

8 Puzzle III

- How about using the *actual cost* as a heuristic?
  - Would it be admissible?
  - Would we save on nodes expanded?
  - What's wrong with it?



- With A*: a trade-off between quality of estimate and work per node
  - As heuristics get closer to the true cost, you will expand fewer nodes but usually do more work per node to compute the heuristic itself

Trivial Heuristics, Dominance

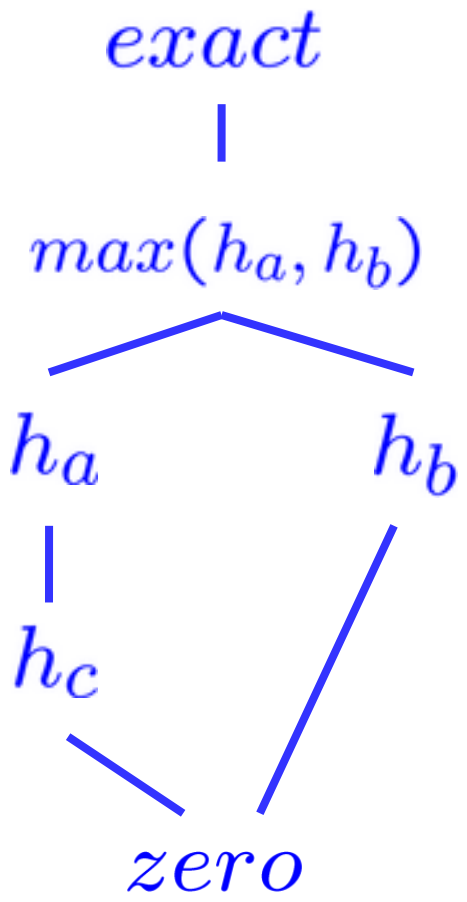
- Dominance: $h_a \geq h_c$ if

$$\forall n : h_a(n) \geq h_c(n)$$

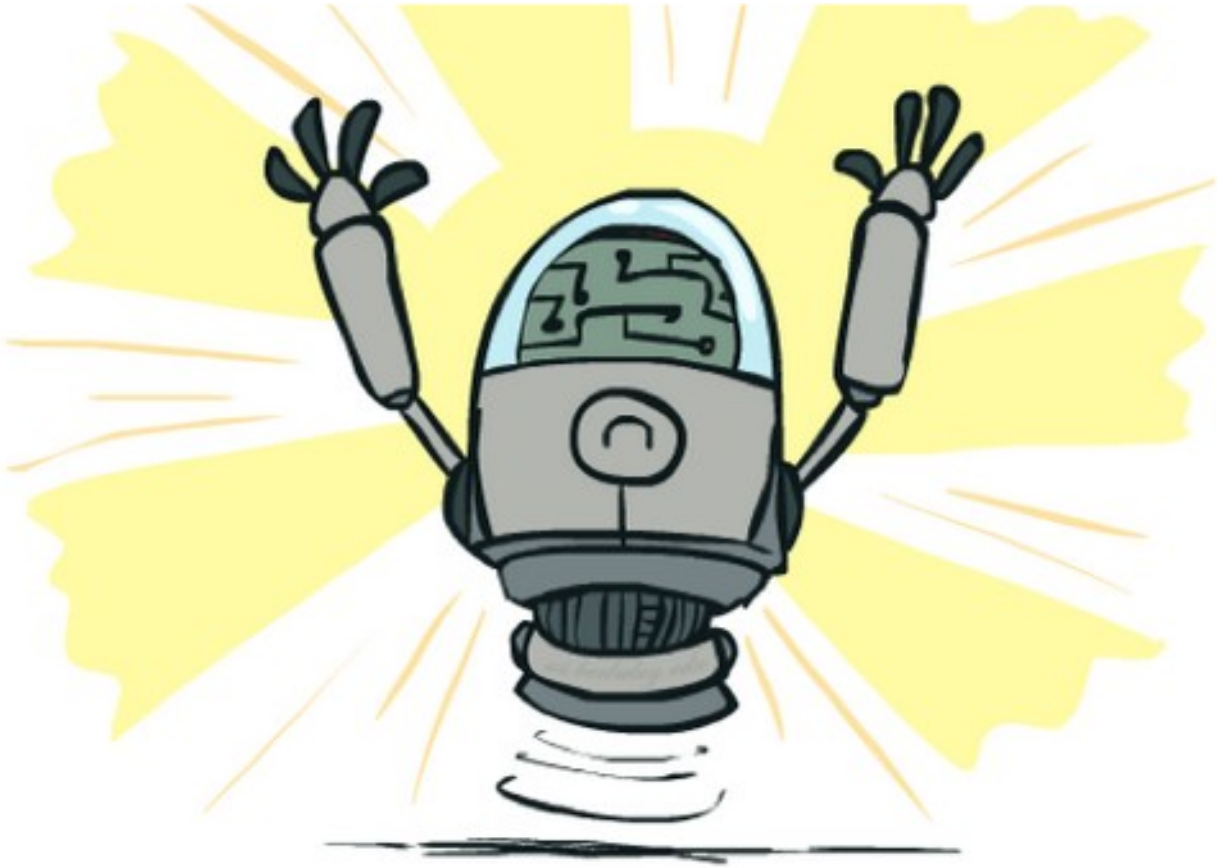
- Heuristics form a semi-lattice:
  - Max of admissible heuristics is admissible

$$h(n) = max(h_a(n), h_b(n))$$

- Trivial heuristics
  - Bottom of lattice is the zero heuristic (what does this give us?)
  - Top of lattice is the exact heuristic

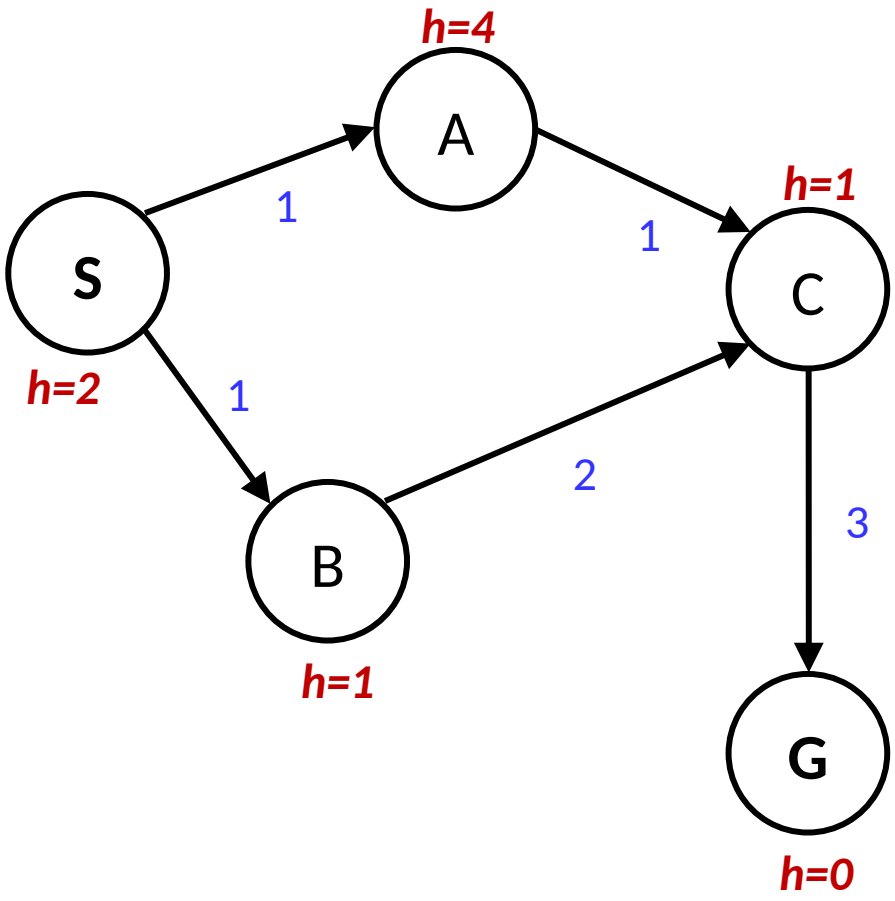


Optimality of A* Graph Search (w. closed set loop breaking)

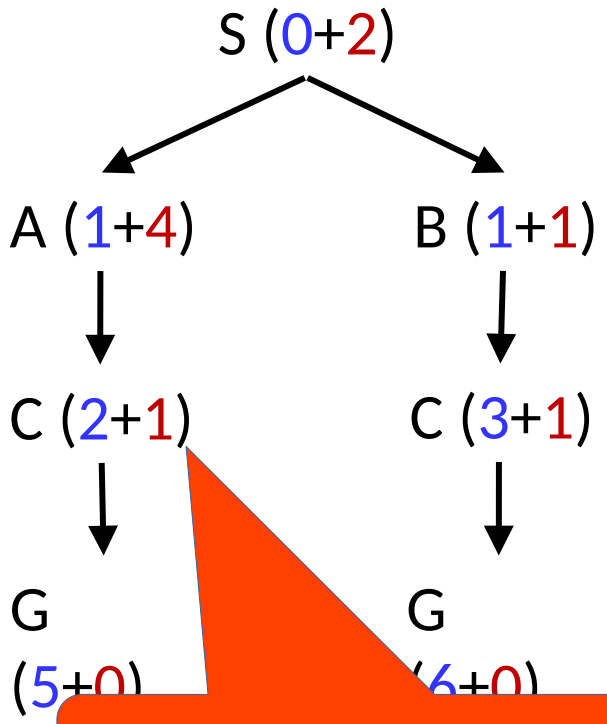


A* Graph Search Gone Wrong?

State space graph

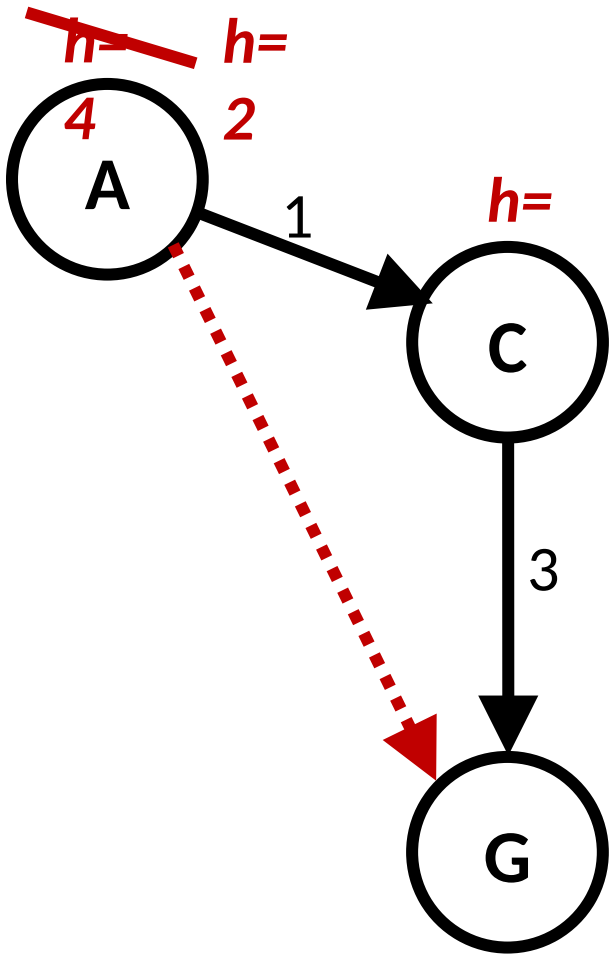


Search tree



Not expanded due to loop breaking:
C already in 'closed' set

Alternative: Consistency of Heuristics



- Main idea: estimated heuristic costs ≤ actual costs
  - Admissibility: heuristic cost ≤ actual cost to goal

    $h(A) \leq h^*(A)$
  - Consistency: heuristic "arc" cost ≤ actual cost for each arc

    $h(A) - h(C) \leq c(A,C)$

    or $h(A) \leq c(A,C) + h(C)$ (=triangle inequality)
    - Note: h* *necessarily* satisfies triangle inequality

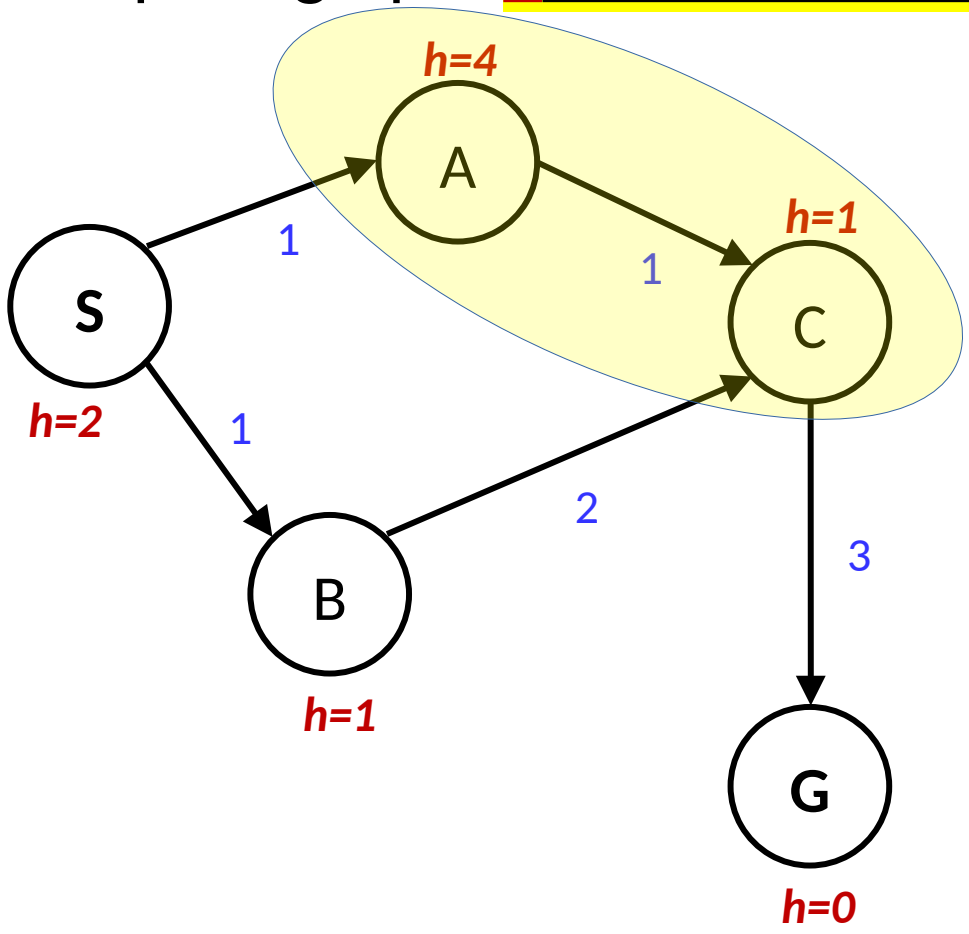
- Consequences of consistency:

The $f(n)$ (=$g(n)$+$h(n)$) value along a path never decreases:

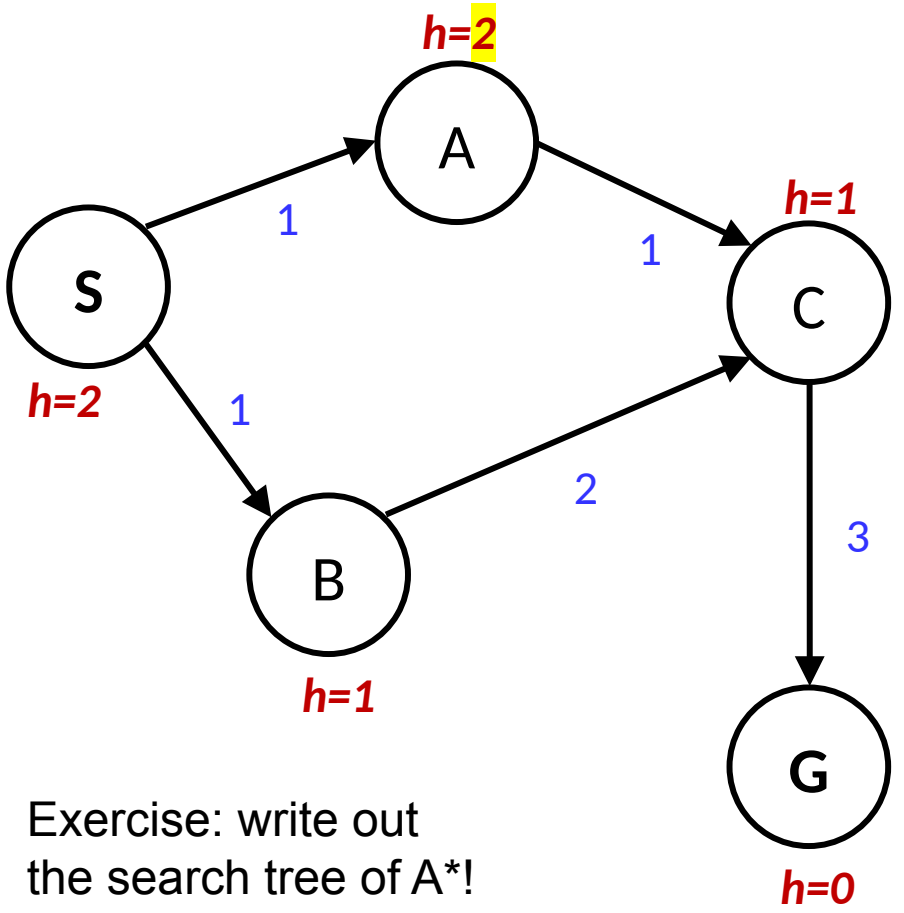
$$h(A) \leq c(A,C) + h(C)$$
$$\Rightarrow g(A) + h(A) \leq g(A) + c(A,C) + h(C)$$
$$\Rightarrow g(A) + h(A) \leq g(AC) + h(C)$$
$$\Rightarrow f(A) \leq f(AC)$$

A* Graph Search Gone Wrong?

State space graph: h not consistent



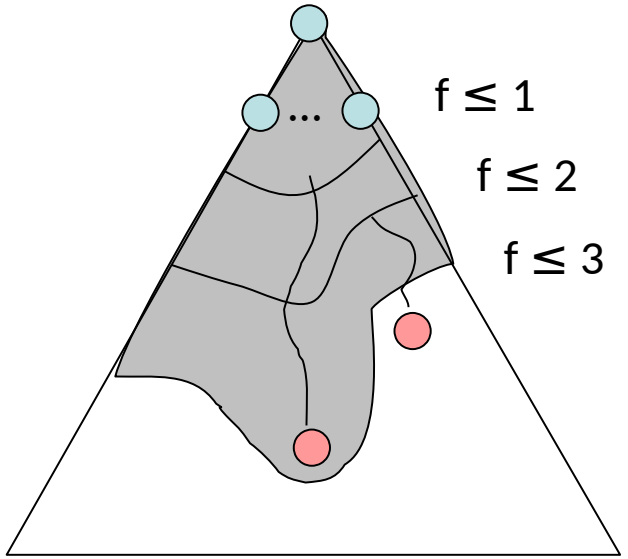
This h would be consistent



Exercise: write out
the search tree of A*!

Optimality of A* Graph Search

- Sketch: consider what A* does with a consistent heuristic:
  - Fact 1: In tree search, A* expands nodes in increasing total f value (f-contours)
  - Fact 2: For every state s, nodes that reach s optimally are expanded before nodes that reach s suboptimally
    (f value along path never decreases)
  - Result: A* graph search is optimal



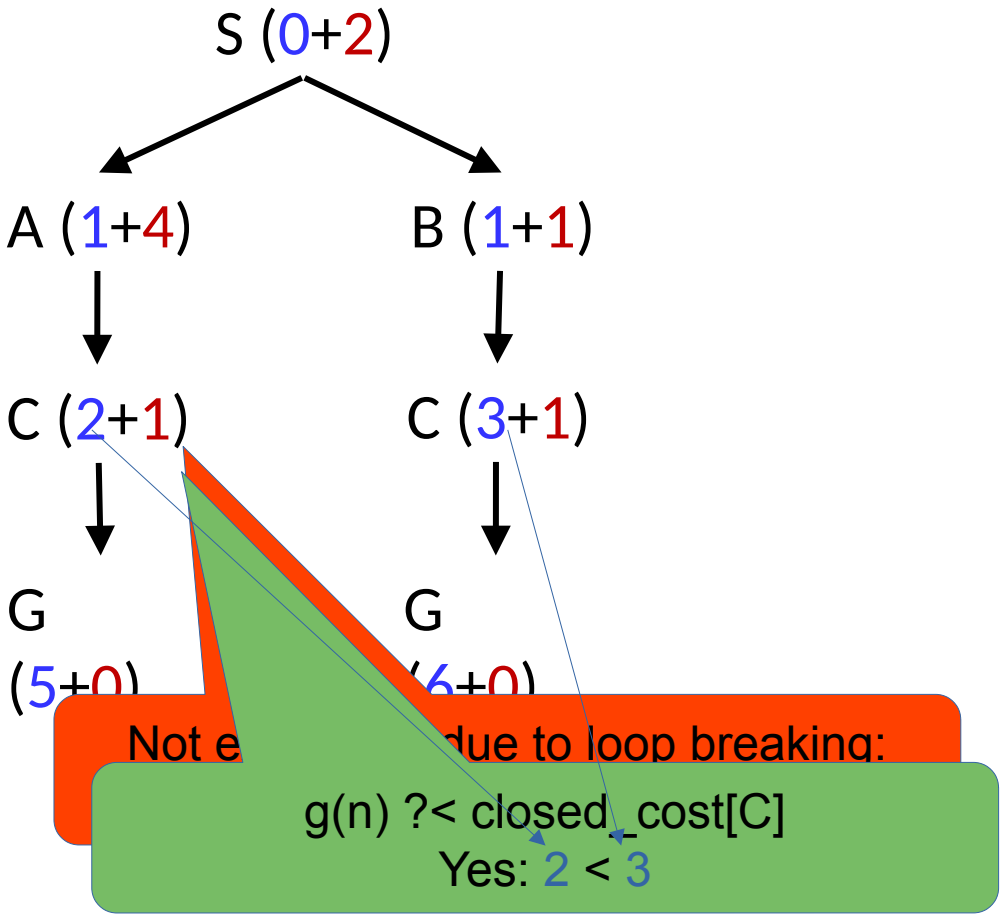
Optimality

- Tree search:
  - A* is optimal if heuristic is admissible
- Graph search:
  - A* with closed *set* is optimal if heuristic is consistent
- Consistency implies admissibility
- Most natural admissible heuristics tend to be consistent, especially if from relaxed problems



Upgrade: optimal A* with admissible heuristic

Search tree



- Closed *set* stores which states expanded
- Closed *dictionary* 'closed_cost' stores which state AND at what cost expanded

New check:

- expand if state *s* not in closed dictionary OR if current-cost g(n) < closed_cost[s]
- Then update closed_cost dictionary

Upside: less restricted when designing heuristics
Downside: A crappy heuristic might make us expand the same state many times…

But…

- A* keeps the entire explored region in memory
- => will run out of space before you get bored waiting for the answer
- There are variants that use less memory (AIMA Section 3.5.5):
  - IDA* works like iterative deepening, except it uses an *f*-limit instead of a depth limit
    - On each iteration, remember the smallest *f*-value that exceeds the current limit, use as new limit
    - Very inefficient when *f* is real-valued and each node has a unique value
  - RBFS is a recursive depth-first search that uses an *f*-limit = the *f*-value of the best alternative path available from any ancestor of the current node
    - When the limit is exceeded, the recursion unwinds but remembers the best reachable *f*-value on that branch
  - SMA* uses *all available memory* for the queue, minimizing thrashing
    - When full, drop worst node on the queue but remember its value in the parent

A*: Summary

- A* orders nodes in the queue by $f(n) = g(n) + h(n)$
- A* is optimal for trees/graphs with admissible/consistent heuristics

- Heuristic design is key: often use relaxed problems

